A Non-Inflatable Coverage Metric for Image-Derived Contour Reconstruction with Continuous-Control Reinforcement Learning

Himanshu Sinha
Department of Electronics and Communication Engineering
Indian Institute of Information Technology, Sri City
India
himanshu.cs24@iiits.in

Atin Chowdhury
Department of Computer Science and Engineering
Indian Institute of Information Technology, Sri City
India
atin.c23@iiits.in

Piyush Joshi
Department of Computer Science and Engineering
Indian Institute of Information Technology, Sri City
India
piyush.j@iiits.in

***Abstract*—Tracing a contour extracted from an image under continuous control requires an objective that distinguishes genuine traversal from mere proximity. In an early version of our environment, a proximity-based reward paid a hand-written raster sweep more than a random policy, although the sweep never followed the target. We address this with a target-pixel-level coverage metric: a per-episode Boolean mask over contour pixels, updated by a swept point-to-segment test, so that coverage is monotone and cannot be increased by revisiting, with reward given only for newly covered pixels. On a synthetic circle the metric reproduces traced arc fractions to within 0.81 percentage points, and under the final reward, raster, stationary and off-contour policies earn negative return. We do not claim that the composite reward is unhackable: its continuity term is not invariant to tracing speed, and a scripted tracer that covers the same ≈95% of the contour earns a return that falls from 856 to 416 as its speed rises from 2 to 12 units/s. We also document the metric's limits: a small end-cap bias, partial credit between parallel edges closer than the coverage tolerance, and a difference below half a percentage point between swept and point-sampled coverage at the speeds our dynamics allow. A Proximal Policy Optimization (PPO) agent with a CNN–MLP policy, taking the final model of each run, reaches $86.4 \pm 4.7\%$ mean coverage over five seeds at a 95% completion threshold, completing 5 of 100 test starts. A single run trained with an 80% threshold (seed 2026) reaches 94.3% and completes 9 of 20 starts; this result rests on one run. However, a greedy controller that steers toward the nearest uncovered pixel, using a feature already present in the agent's observation, reaches the threshold from every start on every target tested, and the learned policies transfer poorly to unseen open strokes. Observation ablations show that removing that feature leaves PPO unchanged, removing the image input improves it (94.1%), and adding sensing noise degrades it (63.3%). We therefore treat the greedy controller as the reference bar for learned contour tracers and report that, in the present observation design, learning is not needed to reach it.**

***Index Terms*—reinforcement learning, proximal policy optimization, PPO, continuous control, contour tracing, curve reconstruction, coverage path planning, reward hacking, reward shaping**

I. Introduction

Many robotic and automation tasks require a controller to follow a geometric trajectory extracted from visual information. Examples include seam following, printed-path inspection, boundary tracing, drawing, and trajectory reproduction. When the target is specified by an image and the controlled agent has inertia, the problem extends beyond discrete path planning: the controller must generate continuous motion that maximizes the portion of the target geometry actually traversed.

Coverage path planning (CPP) has traditionally focused on covering two-dimensional regions of free space [5], [7]. Reinforcement learning has since been applied to coverage planning with both discrete and continuous action spaces [8], [12]. These area-oriented formulations are not designed to quantify traversal of a thin, image-derived contour. The problem considered here is related to CPP but differs in its target: instead of maximizing area coverage, the objective is to trace a one-dimensional contour embedded in a two-dimensional workspace.

This distinction creates an important challenge. A proximity-based reward can assign positive credit whenever an agent passes near a contour, even if the agent never follows the curve. A policy can then learn to exploit the reward rather than solve the intended task, a known issue in reinforcement learning [14]–[16]. Our own first formulation credited 5×5 pixel blocks near the bead; a hand-written raster sweep that drove back and forth across the arena earned $+36.9$ return per episode against $+3.1$ for a random policy, without following the contour. The central question therefore became:

How can contour coverage be defined and rewarded so that the coverage metric directly reflects the portion of the target contour traversed by the agent?

We address this question with three design principles. First, coverage is computed at the resolution of the target contour: every contour pixel has a Boolean covered/uncovered state. Second, coverage is updated using the entire motion segment between two consecutive agent positions rather than only the instantaneous position. Third, reward is derived from newly covered target pixels, while the evaluation metric is computed independently of the reward.

The main contributions of this work are:

1) A continuous-control environment for image-derived contour reconstruction with inertial bead dynamics.
2) A target-resolution swept coverage metric based on a per-episode Boolean mask, with an empirical account of its biases and of its sensitivity to the coverage tolerance.
3) A reward built on that metric under which the degenerate policies we tested (raster sweep, zero thrust, off-contour motion) earn negative return on the training target, together with a direct test showing that its continuity term is not speed-invariant (Section XII).
4) An evaluation of a CNN–MLP PPO agent across seeds, checkpoints, reward ablations, observation ablations and unseen contours, compared under one protocol with random, raster, greedy and pure-pursuit controllers, which establishes a feature-based greedy controller as the reference bar for learned contour tracers.

II. Related Work

A. Coverage Path Planning

Classical coverage path planning seeks trajectories that cover a specified region while minimizing redundant motion. Cellular decomposition and boustrophedon approaches are established techniques for area coverage [5]–[7]. Their natural coverage unit, however, is an area or cell, whereas the present problem requires coverage of a one-dimensional curve. When the target occupies only a small portion of a grid cell, cell-level credit can differ substantially from the length of curve traversed; the proposed coverage operator therefore works directly on contour pixels.

B. Reinforcement Learning for Coverage

Deep reinforcement learning has been applied to coverage planning with value-based and actor-critic algorithms. Theile et al. studied Double-DQN-based UAV coverage under varying power constraints [8]. Garrido-Castaneda et al. investigated A2C and PPO for grid-based coverage planning [9]. Chen et al. proposed a curiosity-driven Re-DQN approach for mowing-robot coverage [10]. Devo et al. studied single-image drone exploration with an asymmetric actor-critic approach [11]. Jonnarth et al. learn continuous coverage paths in unknown environments with multi-scale egocentric maps [12] and study their sim-to-real transfer [13]. These methods reward newly covered area; we apply the same first-visit principle to contour pixels under inertial dynamics, with the target known before the episode begins.

C. Reinforcement Learning for Drawing

Stroke-based rendering with reinforcement learning also reproduces images through actions: SPIRAL [17], model-based painting agents [18] and sketching with deep Q-networks [19]. These agents place strokes; ours must move a body with momentum along a fixed target.

D. Reward Hacking and Shaping

Reward hacking occurs when an agent maximizes the specified reward while failing to satisfy the intended objective [14]–[16]. In coverage problems, this can occur when proximity to a target is rewarded instead of actual traversal. Potential-based shaping preserves optimal policies [26]; our approach-shaping term is not potential-based (Section VII), so its effect is examined empirically. Rather than attempting to prove that a dense reward is unhackable, we construct the coverage metric so that repeated traversal cannot increase it, and then test specific degenerate policies.

III. Problem Formulation

The contour reconstruction task is formulated as an episodic Markov decision process

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma). \tag{1}$$

The target contour is represented by a set of pixels

$$\mathcal{C} = \{q_k\}_{k=1}^{N}, \quad q_k \in \mathbb{Z}^2, \tag{2}$$

where N is the number of contour pixels. The agent produces a continuous planar action $a_t \in [-1, 1]^2$, a normalized two-dimensional thrust vector. The objective is to maximize the coverage

$$C_t = \frac{1}{N} \sum_{k=1}^{N} m_{t,k}, \tag{3}$$

where $m_{t,k} \in \{0, 1\}$ indicates whether contour pixel q_k has been covered by time t (Section VI). Because a mask element can transition only from zero to one,

$$C_{t+1} \geq C_t. \tag{4}$$

IV. Target Extraction

The input image I is resized to an 800×800 working raster, converted to grayscale, smoothed with a 5×5 Gaussian kernel, and passed to the Canny detector [21]:

$$E = \mathrm{Canny}\big(G_{5\times5} * \mathrm{Gray}(I),\, 50,\, 150\big). \tag{5}$$

Contours are then extracted by border following [22] using OpenCV [23]. Contours with closed arc length below 30 pixels are discarded as noise. The remaining contours are rasterized into the target set $\mathcal{C}$, and the Euclidean distance transform [24], [25]

$$D(x) = \min_{q \in \mathcal{C}} \|x - q\|_2 \tag{6}$$

is computed once per target.

A drawn stroke of non-negligible width produces two approximately parallel Canny edges, one on each side of the

TABLE I
ENVIRONMENT CONFIGURATION

Parameter	Value
Arena	$[-10, 10]^2$ world units
Bead radius	0.55
Acceleration α	18.0
Maximum speed $v_{\max}$	15.0 units/s
Drag coefficient β	5.0
Boundary restitution	0.45
Timestep Δt	1/60 s
Coverage tolerance ρ	8 px
Episode limit	2500 steps
Stagnation limit	350 steps without new coverage

line. The primary target (`path1`, Fig. 1) is a single closed stroke; its $N = 4{,}938$ target pixels form two parallel closed edges about 8.4 px apart. Border following reports each edge twice, giving four contour entries; these do not correspond to four distinct shapes. Every target is resized to 800×800 without preserving its aspect ratio, so the two open strokes are compressed horizontally.

V. CONTINUOUS-CONTROL ENVIRONMENT

The simulated bead is a disc of radius 0.55 world units moving in the square arena $[-10, 10]^2$, which maps onto the raster at 39.95 px per world unit. The control period is $\Delta t = 1/60$ s. Table I lists the parameters.

The action is projected onto the unit disk, $\tilde{a}_t = a_t / \max(1, \|a_t\|_2)$, with thrust magnitude $\mu_t = \|\tilde{a}_t\|_2$. The velocity and position are then updated as

$$v'_t = v_t + \alpha\, \tilde{a}_t\, \Delta t, \tag{7}$$

$$v''_t = v'_t - (1 - \mu_t)\, \beta\, v'_t\, \Delta t, \tag{8}$$

$$v_{t+1} = v''_t\, \min\big(1,\, v_{\max}/\|v''_t\|_2\big), \tag{9}$$

$$p_{t+1} = p_t + v_{t+1}\, \Delta t. \tag{10}$$

Drag therefore vanishes at full thrust and is largest at zero thrust. The bead's centre is confined to $[-9.45, 9.45]^2$ (the arena minus the bead radius), and boundary collisions reflect the normal velocity component with restitution 0.45. At $v_{\max}$ the bead would move about 10 pixels per step; the trained policies operate far below this speed (Section XVI). An episode terminates successfully when $C_t \geq \phi$, and is truncated after 2500 steps or 350 consecutive steps without new coverage.

VI. SWEPT-CAPSULE COVERAGE METRIC

A. Definition

Let the bead's pixel positions at consecutive steps be f_{t-1} and f_t, and $s_t = [f_{t-1}, f_t]$ the swept segment. For contour pixel q_k, the closest point on the segment is $p^* = f_{t-1} + \tau^*(f_t - f_{t-1})$ with

$$\tau^* = \mathrm{clip}\left(\frac{(q_k - f_{t-1})^\top (f_t - f_{t-1})}{\|f_t - f_{t-1}\|_2^2},\, 0,\, 1 \right), \tag{11}$$

and $d(q_k, s_t) = \|q_k - p^*\|_2$. With tolerance $\rho = 8$ pixels, the set of newly covered pixels and the mask update are

$$H_t = \{k :\ m_{t-1,k} = 0 \ \wedge\ d(q_k, s_t) \leq \rho\}, \tag{12}$$

$$m_{t,k} = m_{t-1,k} \ \vee\ \mathbf{1}[k \in H_t], \quad n_t = |H_t|. \tag{13}$$

Since previously covered pixels cannot become newly covered again, repeated traversal cannot increase the metric. The same operator runs once at reset on the spawn point, without reward, so the bead's own footprint is not paid for.

B. Motivation for the Swept Test

If coverage were determined only from the bead's position at the end of each step, a fast agent could cross a contour between two samples. The swept test removes this possibility for any step length. In this environment the maximum step ($\approx$10 px) is smaller than the band width $2\rho = 16$ px, so full tunnelling cannot occur and the practical difference is small (Section XI).

VII. REWARD DESIGN

The reward consists of coverage, continuity, approach-shaping, smoothness, far-field, anti-oscillation, and completion components:

$$R_t = R_t^{\mathrm{cov}} + R_t^{\mathrm{cont}} + R_t^{\mathrm{app}} - R_t^{\mathrm{sm}} - R_t^{\mathrm{far}} - R_t^{\mathrm{osc}} + R_t^{\mathrm{done}}. \tag{14}$$

A. Coverage Reward

$$R_t^{\mathrm{cov}} = \lambda_{\mathrm{cov}}\, \frac{100\, n_t}{N}, \qquad \lambda_{\mathrm{cov}} = 4.0, \tag{15}$$

i.e., 4 per percentage point of new coverage and at most 400 per episode. Because n_t counts only first-time coverage, it cannot be earned twice for the same pixels.

B. Continuity Reward

$$R_t^{\mathrm{cont}} = \lambda_{\mathrm{cont}}\, \frac{n_t^c}{n_t} \ \ (n_t > 0), \qquad \lambda_{\mathrm{cont}} = 0.15, \tag{16}$$

where n_t^c counts newly covered pixels within 2 px of a pixel that was already covered; the term is 0 when $n_t = 0$. Unlike the coverage term, this term is paid on every coverage-producing step, so it is not invariant to how finely the same pixels are covered (Section XII).

C. Approach Shaping

Let u_t be the distance from the bead to the nearest uncovered contour pixel. When no new coverage occurs,

$$R_t^{\mathrm{app}} = \mathbf{1}[n_t = 0]\ \mathrm{clip}\big(\kappa(u_{t-1} - u_t),\, -\delta,\, \delta\big), \tag{17}$$

with $\kappa = 0.015$ and $\delta = 0.3$. The term is suppressed on coverage-producing steps so that it does not pay twice for the same event. Because it is gated and its reference pixel changes whenever coverage occurs, it is not potential-based [26]; the clip at δ is inactive at the maximum step of about 10 px, since $\kappa \cdot 10 = 0.15 < \delta$. Its effect is therefore evaluated empirically through reward ablations.

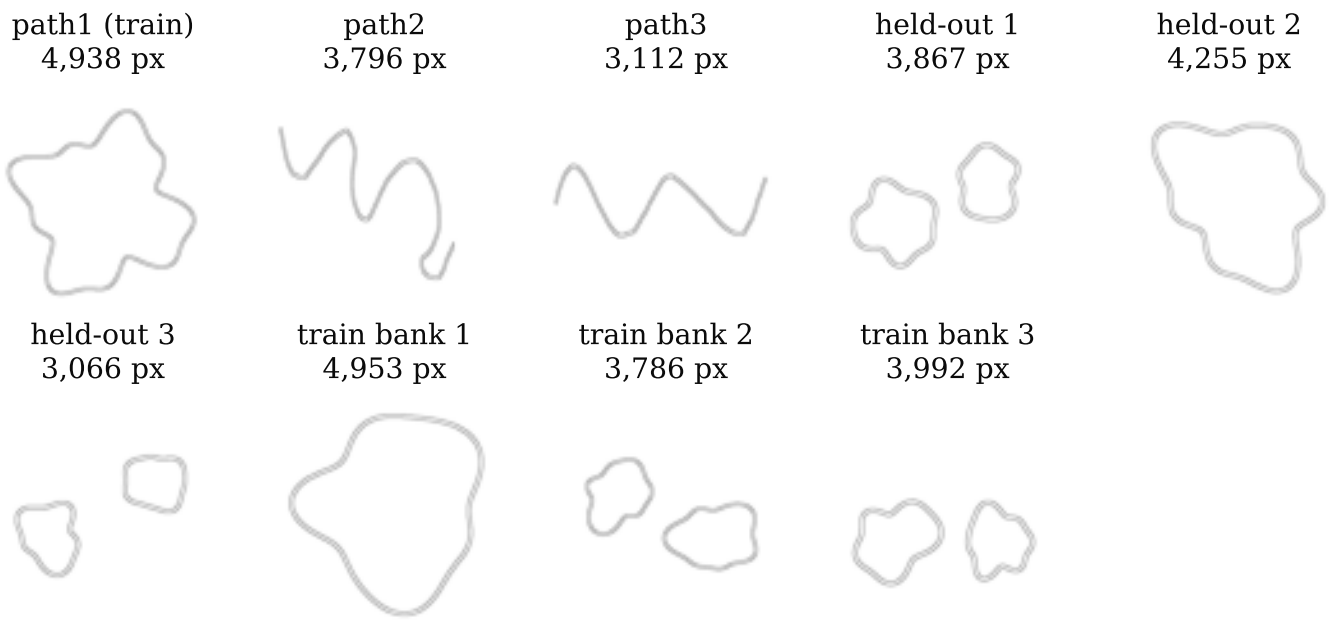


Fig. 1. Target contours after edge extraction, with target pixel counts. `path1` is the primary training target; `path2` and `path3` are unseen open strokes; "held-out" shapes are procedurally generated test contours; "train bank" shows samples from the procedural training distribution.

D. Action Smoothness

$$R_t^{\mathrm{sm}} = \min\big(\sigma \,\|\tilde{a}_t - \tilde{a}_{t-1}\|_2^2,\, \sigma_{\max}\big), \tag{18}$$

with $\sigma = 0.02$ and $\sigma_{\max} = 0.05$.

E. Far-Field Penalty

With $g_t = D(f_t)$ the distance to any contour pixel,

$$R_t^{\mathrm{far}} = \mathbf{1}[g_t > \theta]\ \min\big(\eta(g_t - \theta)^2,\, \eta_{\max}\big), \tag{19}$$

with $\theta = 50$ px, $\eta = 2 \times 10^{-4}$ and $\eta_{\max} = 0.15$.

F. Anti-Oscillation Penalty

The raster is divided into 16×16 px cells. Let e_t indicate that the bead re-entered a cell it had left within the last 60 steps. Then

$$R_t^{\mathrm{osc}} = \lambda_{\mathrm{osc}}\, \mathbf{1}\big[n_t = 0 \ \wedge\ (e_t \ \vee\ \|f_t - f_{t-1}\|_2 < 1)\big], \tag{20}$$

with $\lambda_{\mathrm{osc}} = 0.05$ (the second condition catches a bead that barely moves).

G. Completion Reward

A bonus of 100 is paid once, at the first step with $C_t \geq \phi$, which also ends the episode:

$$R_t^{\mathrm{done}} = 100\, \mathbf{1}[C_t \geq \phi \ \wedge\ C_{t-1} < \phi]. \tag{21}$$

All results use $\phi = 0.95$ unless stated.

VIII. Observation Space and Policy Architecture

A. Visual Observation

The visual observation is a $64 \times 64 \times 1$ crop, centred on the bead, of the distance map normalized as $\min(D/100, 1)$ and stored as 8-bit values. The crop is cut from the 800×800 raster without rescaling, so it spans 64 px, about 1.6 world units or roughly six maximum-speed steps across. Covered contour pixels are erased from this map (their 5×5 neighbourhood is set to the background value), so the crop shows only the part of the target that remains to be traced.

B. Numerical State

The numerical state contains 11 normalized quantities:

$$\begin{aligned} o_t^s = \big[&p_x/10,\ p_y/10,\ v_x/v_{\max},\ v_y/v_{\max},\\ &\hat{u}_x,\ \hat{u}_y,\ u_t/100,\ g_t/100,\\ &2C_t - 1,\ \tilde{a}_{x,t-1},\ \tilde{a}_{y,t-1}\big], \end{aligned} \tag{22}$$

where $\hat{u}$ is the unit vector toward the nearest uncovered contour pixel and the two distances are clipped to 1. The vector $\hat{u}$ and the distance u_t are computed from the coverage mask, not from the erased image.

C. Observation Tiers

To test which inputs the policy uses, we define four observation tiers; the observation shapes, dynamics, reward and coverage computation are identical in all of them. *Full* is the observation described above. *No-direction* sets the entries $\hat{u}_x$, $\hat{u}_y$ and $u_t/100$ to zero. *No-direction-noisy* additionally adds Gaussian noise to the observed position ($\sigma = 0.05$ world units) and velocity ($\sigma = 0.05\, v_{\max}$); the noise affects the observation

TABLE II
CONVOLUTIONAL LAYERS OF THE VISUAL BRANCH

Input ch.	Output ch.	Kernel	Stride	Padding
1	16	5×5	2	2
16	32	3×3	2	1
32	32	3×3	2	1

TABLE III
PPO TRAINING CONFIGURATION

Parameter	Value
Learning rate	3×10^{-4}
Rollout length $\times$ environments	1024×4
Batch size / epochs per rollout	256 / 10
γ / GAE λ	0.995 / 0.95
Clip range	0.2
Value / entropy coefficient	0.5 / 0.005
Gradient clipping	0.5
Total timesteps	1,000,000
Seeds (full, no-direction)	42, 1, 2, 3, 4
Seeds (noisy, no-visual)	42, 1, 2
Seed ($\phi = 0.80$ run)	2026

only. *No-visual* keeps the full numerical state and replaces the image crop with zeros.

D. CNN–MLP Feature Extractor

The visual branch has three convolutional layers with ReLU activations (Table II). The resulting $32 \times 8 \times 8 = 2048$ features are projected to 128 dimensions. The numerical branch has two fully connected layers of 64 units. The two branches are concatenated and projected to a shared 256-dimensional representation, followed by separate policy and value heads of two 128-unit layers each. The network has about 430,000 parameters.

IX. PPO TRAINING

The agent is trained with Proximal Policy Optimization [1] and Generalized Advantage Estimation [4], implemented in Stable-Baselines3 [2] and Gymnasium [3]. Truncated episodes bootstrap from the value function; only successful completion is treated as terminal. Table III gives the configuration. Training episodes start at a random contour pixel with up to ± 0.12 world units of jitter.

X. EXPERIMENTAL PROTOCOL

Every method runs in the same environment with the same coverage code and $\phi = 0.95$. Test starts are 20 contour pixels evenly spaced in the raster-scan (row-major) order of the contour-pixel table, without jitter; PPO acts deterministically. Validation starts are 20 further pixels interleaved between the test starts (19 half-way between consecutive test starts and one at a quarter gap); they are used, and the test starts never are, to select checkpoints and tune controller speeds. Besides `path1` we evaluate on two unseen open strokes and ten procedurally generated closed shapes (radial Fourier curves drawn as strokes, from a seed disjoint from any training bank). We report mean $\pm$ standard deviation over starts, completion count, mean return and mean episode length (which includes truncated episodes), and the spread of seed means across seeds (sample standard deviation).

Model selection. Unless stated otherwise, every PPO result is the final model saved when training ends, with no checkpoint selection. Because the last 4096-step rollout is completed in full, training stops at 1,003,520 steps rather than exactly 10^6. Checkpoints are also written every 50k steps; Table XI evaluates every second one. A validation-selected checkpoint (earliest checkpoint wins ties) is reported only as a sensitivity analysis (Section XV).

Controllers. Random samples actions uniformly; zero thrust never moves; raster sweeps the arena in horizontal lanes. Greedy steers toward the nearest uncovered contour pixel with a velocity controller at 5 units/s; its inputs are $\hat{u}$ and the bead's own velocity, both already part of the agent's state. Pure pursuit [20] is given the ordered contour polylines, traces the nearest untraced one from its closest point, and skips polylines that are already 90% covered, at 5 units/s. Both speeds were tuned on the validation starts of `path1` over $\{3, 4, 5, 6, 8, 10, 12\}$; to check that the controllers are not simply being run slower than the learned policy, both were also evaluated at 15 units/s ($v_{\max}$). Measured mean speeds are reported in Table VIII.

Software. Results were produced with PyTorch 2.13, Stable-Baselines3 2.9, Gymnasium 1.1 and OpenCV 5.0; all evaluation numbers are CPU, one torch thread. Rollouts are chaotic: the same seed-42 final model scores 86.03% on `path1` on CPU with one thread, 85.99% on CPU with default threads and 82.03% on a GPU, so the evaluation setting must be stated with any number. The seed-42 rows of Tables VII, X and XI were produced on the original machine and agree with the released evaluation harness to about 0.15 percentage points on `path1`; all other rows come from the released harness.

The $\phi = 0.80$ run. The $\phi = 0.80$ run reported in Table VII was repeated with a recorded seed (2026; the PPO seed and the four environment seeds are 2026–2029). It uses the configuration of Tables I–III unchanged except for the completion threshold, was trained on `path1` only for 1,003,520 steps (3,541 s, CPU, Windows 11, Python 3.13.9), and is evaluated as the final model at $\phi = 0.95$ on all 13 targets with the same harness and test starts as every other row, with no checkpoint selection.

The environment parameters, reward coefficients, network architecture and PPO hyperparameters are fully specified in Tables I–III and Sections V–IX. The code, pinned requirements, raw per-start results and the scripts that regenerate every table are available in the project repository.[1]

XI. VALIDATION OF THE COVERAGE METRIC

Arc fractions. On a synthetic circle of circumference $L \approx 1{,}634$ px, a pure-pursuit controller traced fixed fractions of the arc. Tracing a fraction f also credits pixels within ρ of each end, so the expected value lies between f and

[1] https://github.com/himanshu09120/bead-tracer

TABLE IV
COVERAGE AFTER TRACING A FRACTION OF A CIRCLE ($L \approx 1{,}634$ PX, $\rho = 8$ PX)

Traced arc f	Measured	Upper bound $f + 2\rho/L$
5%	5.50%	5.98%
10%	10.36%	10.98%
25%	25.20%	25.98%
50%	50.18%	50.98%
75%	75.81%	75.98%

$f + 2\rho/L$. Table IV shows measured values in that range, within 0.81 percentage points of f at every fraction tested. A full traversal reaches 100%, re-tracing an already covered arc four times adds 0.00 percentage points, and a policy that never touches the contour stays at 0%. The last three follow from the construction and serve as tests of the implementation.

Parallel edges. Because the two edges of the `path1` stroke are 8.4 px apart, only slightly more than ρ, a controller that traces exactly one edge is also credited with 26.8% of the other (63.0% total). Tracing the stroke's centre line covers both edges. The metric therefore measures coverage of the stroke, not of each edge separately, whenever edges are closer than about 2ρ; for the same reason a trajectory can cut across features narrower than 2ρ.

Swept versus point-sampled coverage. Recomputing coverage from end-of-step positions only, on the PPO agent's own trajectories from the 20 test starts, gives 85.95% against 86.18% with the swept test (largest per-episode difference 0.41 percentage points). The swept test is the correct definition, but at these speeds it changes results by less than half a percentage point.

Sensitivity to the tolerance. Table V recomputes the coverage of the same trajectories under tolerances from 2 to 16 px (training and all other results use $\rho = 8$). The absolute values depend strongly on ρ: at $\rho = 2$ pure pursuit reaches 78.5% and the five-seed PPO mean about 17%, while at $\rho = 16$ both exceed 96%. The ordering of the methods is largely stable: greedy, pure pursuit and the no-visual PPO agent stay ahead of the final full-observation PPO models at every tolerance from 2 to 16 px. The 95% completion threshold is therefore a statement about $\rho = 8$, and coverage values should not be compared across tolerances.

XII. REWARD HACKING AND DEGENERATE POLICIES

Three degenerate behaviours were tested under the final reward on the primary target (Table VII, 20 test starts): a raster sweep covers 7.0% with mean return -29.3; zero thrust covers 0.4%, all of it the unrewarded spawn footprint, with return -17.5; uniformly random actions cover 2.3% with return -38.6. On the synthetic circle, an off-contour jitter policy covers 0.00% and earns -60.66, and over an equal 400-step budget honest tracing earns $+390.25$ (94.01% coverage) against -12.72 (10.49%) for the raster sweep. Under the earlier block-based reward the same raster sweep had outearned a random policy.

These results show that the strategies tested are not profitable; they do not establish that the reward cannot be gamed by other policies.

Limits of the reward: a slow-tracer test. The guarantees above concern the coverage metric and the coverage term R^{cov}, which pays once per pixel. The continuity term does not: it pays up to $\lambda_{\text{cont}} = 0.15$ on every step with $n_t > 0$, so a policy that covers the same pixels in more, smaller increments should earn more total reward at identical coverage. We tested this directly with a scripted pure-pursuit tracer run at fixed speeds from 2 to 12 units/s on `path1` (10 test starts, step cap raised to 20,000 so that slow tracers can finish). Table VI shows that coverage stays at about 95% and the coverage reward is nearly constant (377–379), while the continuity reward falls from 402.9 at 2 units/s to 53.0 at 12 units/s and the total return from 856.0 to 415.7. The same traced pixels therefore earn about twice the return when traced slowly, and at 2 units/s the continuity reward alone exceeds the coverage reward. We therefore claim non-inflatability for the coverage metric and the coverage term, not for the composite reward, and recommend that comparisons between methods be made on coverage and completion rather than on return.

XIII. RESULTS

A. Primary Evaluation

Table VII gives the main comparison. The seed-42 PPO model (the final model of the run) reaches $86.2 \pm 11.4\%$ mean coverage, with individual episodes from 37.7% to 94.2% (Fig. 2), and does not reach the completion threshold from any start. Selecting its checkpoint on validation starts instead (400k steps, Section XV) gives $90.2 \pm 3.7\%$ and 4 of 20 completions. Across five seeds the final models' mean coverages range from 80.3% to 91.9% ($86.4 \pm 4.7\%$, sample standard deviation of the seed means, 5 completions out of 100 starts), so a single seed is not representative: the seed means span more than 11 percentage points and the completion count varies from 0 to 4 of 20.

PPO clearly learns to trace: it is far above random actions (2.3%) and the raster sweep (7.0%). However, the greedy controller reaches the threshold from every start, in 1817 steps on average, and privileged pure pursuit does the same. The controllers do depend on their speed: at 15 units/s greedy reaches only 57.9% and pure pursuit 46.8%, so the tuned speed is part of the reference bar, and Table VIII reports it next to the measured speeds of the learned policy.

The learned policies do not move at top speed. Their mean speed is about 2.6 units/s, slightly below that of the tuned controllers, and essentially no step exceeds 90% of $v_{\max}$ (Section XVI). Episode lengths are therefore comparable to those of the controllers, and the learned policies do not trace faster.

The strongest learned policy is a single run trained with $\phi = 0.80$, taken as the final model of that run with no checkpoint selection. Its seed, 2026, is recorded (Section X). Evaluated at $\phi = 0.95$ it reaches $94.3 \pm 1.1\%$ and completes the task from 9 of 20 starts, with a mean episode length of 915 steps (this

TABLE V
COVERAGE (%) ON PATH1 TEST STARTS RECOMPUTED AT DIFFERENT TOLERANCES ρ (PX). PPO ROWS ARE SEED MEANS OF FINAL MODELS UNLESS MARKED VAL-SELECTED. †SPEED TUNED ON VALIDATION STARTS.

Method	$\rho = 2$	$\rho = 4$	$\rho = 6$	$\rho = 8$	$\rho = 10$	$\rho = 12$	$\rho = 16$
Pure pursuit†	78.5	88.4	92.7	95.1	98.6	99.1	99.4
Greedy†	51.7	76.1	89.3	95.0	97.9	99.1	99.9
PPO no-visual (val-selected, 3 seeds)	35.4	66.9	87.5	94.9	97.0	98.8	99.7
PPO full (val-selected, 5 seeds)	15.8	44.6	76.3	91.1	95.6	98.1	99.5
PPO full (final, 5 seeds)	17.4	44.1	71.1	86.5	91.9	94.6	96.8
Raster sweep	1.9	3.5	5.2	7.0	8.8	10.6	14.0
Random actions	0.8	1.4	1.8	2.3	2.9	3.4	4.2

TABLE VI
SLOW-TRACER TEST ON PATH1: SCRIPTED PURE-PURSUIT TRACER AT FIXED SPEED (10 TEST STARTS)

Speed	Cov. (%)	Compl.	Return	Cov. rew.	Cont. rew.
2	94.5	9/10	856.0	376.5	402.9
3	95.0	10/10	692.6	378.6	229.8
4	95.0	10/10	601.1	378.6	140.9
6	95.1	10/10	564.0	378.6	99.9
8	95.1	10/10	519.1	378.8	74.7
10	95.1	10/10	468.6	378.7	60.3
12	95.2	10/10	415.7	379.0	53.0

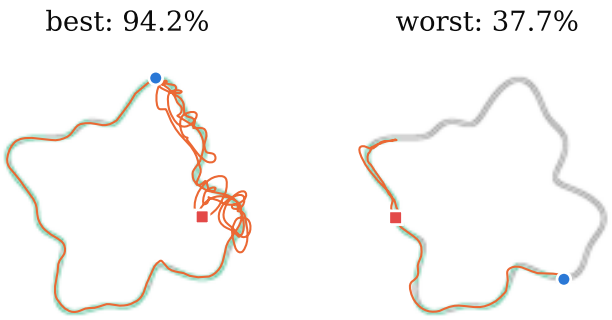


Fig. 2. Best (94.2%) and worst (37.7%) of the 20 test episodes of the seed-42 PPO model on path1. Grey: uncovered target; green: covered; orange: trajectory; circle: start; square: end.

figure includes episodes truncated by the stagnation limit and is not directly comparable to the controllers' lengths). It stays on the contour but does not always pick up the last few percent. An earlier run with the same configuration, whose seed was not recorded, was evaluated with the same harness and gives 94.9% and 13 of 20 completions (94.8% and 12 of 20 in our original evaluation, the difference being consistent with the chaotic rollouts noted in Section X). The two runs agree on coverage but differ by three to four completions; with one run each, we do not attribute that difference to any cause. Both $\phi = 0.80$ runs reach higher path1 coverage than any of the five $\phi = 0.95$ seed means (80.3–91.9%); a plausible explanation is that the lower training threshold ends episodes before the agent enters the loop-and-truncate regime (Section XVI), so less of its experience comes from that regime. This is a comparison of two runs against five and needs more seeds to confirm.

B. Observation Ablations

Table IX evaluates the four observation tiers of Section VIII-C on all targets. Three results stand out. First, removing $\hat{u}$ and u_t does not change the final-model coverage on path1 (86.4% for both tiers), so the policy either does not rely on those inputs or recovers the direction from the image. Second, removing the image helps: the no-visual agent reaches 94.1% on path1 (39 of 60 completions), 91.8% on path2, 90.2% on path3 and 94.9% on the held-out shapes, with much smaller spread across seeds; Section XVI examines why. Third, adding sensing noise on top of removing the direction features is the only change that clearly hurts, to 63.3% on path1. With checkpoint selection on validation starts the picture is similar (91.1%, 92.6%, 60.9% and 94.9% on path1). The no-visual agent keeps $\hat{u}$, so it effectively learns a feedback law on the direction vector, which is the same information the greedy controller uses; it does not exceed that controller. The noisy tier has not been compared with the greedy controller under the same noise, so we cannot say whether its drop reflects a limit of learning or of the task.

XIV. GENERALIZATION TO UNSEEN CONTOURS

Table X evaluates each policy on every target. Policies trained on path1 alone transfer poorly to the two open strokes: the five-seed full-observation mean is 49.9% on path2 and 49.3% on path3, against 86.4% on path1. Transfer to the unseen closed shapes is better and varies strongly between seeds (per-seed means over the ten held-out shapes range from 70.9% to 87.0%). These held-out shapes come from the released generator and are smoother than path1. The $\phi = 0.80$ model of Table VII (seed 2026) scores 55.4% on path2, 61.2% on path3 and $90.9 \pm 0.9\%$ on the ten held-out shapes when evaluated at $\phi = 0.95$; the earlier $\phi = 0.80$ run with an unrecorded seed scores 57.8%, 63.8% and $85.2 \pm 1.2\%$. Both are single runs, so the difference between them, and the comparison of either with the five $\phi = 0.95$ seeds, is not interpretable. Training on a bank of 64 procedural shapes (last row, one seed) improves coverage on the two open strokes relative to the path1-only models, but it is much worse on path1, which it never saw. That model was not re-evaluated on the current held-out shapes because its checkpoint is not part of the released repository, so its held-out cell is left empty. The greedy controller needs no training and is unaffected by the change of target.

TABLE VII
PRIMARY TARGET (PATH1), 20 TEST STARTS, $\phi = 0.95$. COVERAGE IS MEAN $\pm$ SD OVER STARTS (%). PPO ROWS ARE FINAL MODELS UNLESS STATED. †SPEED TUNED ON VALIDATION STARTS.

Method	Coverage (%)	Completed	Return	Steps
Random actions	2.3 ± 1.7	0/20	-38.6	603
Zero thrust	0.4 ± 0.1	0/20	-17.5	350
Raster sweep	7.0 ± 3.7	0/20	-29.3	554
Greedy nearest-uncovered† (5 units/s)	95.0 ± 0.0	20/20	577.0	1817
Pure pursuit (privileged)† (5 units/s)	95.1 ± 0.0	20/20	570.9	1744
Greedy at $v_{\max}$ (15 units/s)	57.9 ± 5.0	0/20	252.0	2500
Pure pursuit at $v_{\max}$ (15 units/s)	46.8 ± 5.7	0/20	132.7	2500
PPO, seed 42, $\phi = 0.95$	86.2 ± 11.4	0/20	401.7	1892
PPO, seed 42, $\phi = 0.95$ (validation-selected, 400k)	90.2 ± 3.7	4/20	489.3	1591
PPO, single run (seed 2026), trained with $\phi = 0.80$	94.3 ± 1.1	9/20	462.0	915
PPO, five seeds, $\phi = 0.95$ (SD of the five seed means)	86.4 ± 4.7	5/100	421.4	1600
PPO, five seeds, $\phi = 0.95$ (validation-selected, SD of seed means)	91.1 ± 1.6	15/100	467.4	1580

TABLE VIII
SPEED-MATCHED COMPARISON ON PATH1 (20 TEST STARTS). MEAN SPEED IS MEASURED IN WORLD UNITS/S ($v_{\max} = 15$).

Method	Cov. (%)	Compl.	Steps	Mean speed
PPO full (final, 5 seeds)	86.4	5/100	1600	2.63
PPO full (val-sel., 5 seeds)	91.1	15/100	1580	2.64
Greedy, 5 units/s	95.0	20/20	1817	3.86
Greedy, 15 units/s	57.9	0/20	2500	4.20
Pure pursuit, 5 units/s	95.1	20/20	1744	4.00
Pure pursuit, 15 units/s	46.8	0/20	2500	5.11

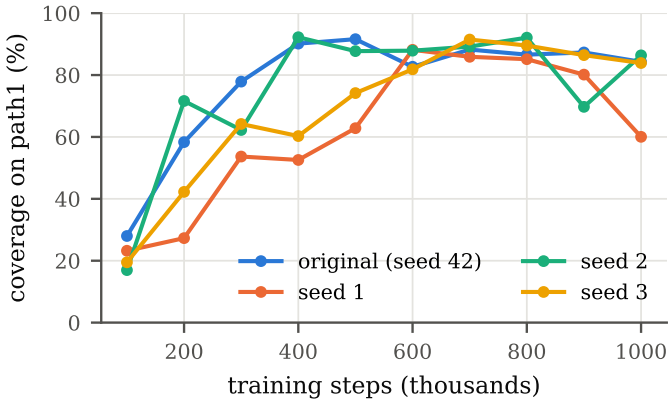


Fig. 3. Deterministic coverage on the path1 test starts across training checkpoints, per seed.

XV. TRAINING DYNAMICS AND CHECKPOINT SELECTION

Deterministic coverage of the seed-42 run changes non-monotonically across checkpoints (Table XI, Fig. 3). Selecting on validation starts picks the checkpoint at 400k steps, which scores 90.2% on the test starts, against 86.2% for the final model and 84.3% for the 1000k checkpoint. These last two differ because the 1000k checkpoint is written part-way through the final rollout, whereas the final model is saved after that rollout completes. Picking the best checkpoint on the test starts themselves (500k, 91.6%) would overstate performance. Checkpoint selection should therefore rely on deterministic coverage on dedicated validation starts, not on training return or on the best score observed on the test set, which should remain reserved for the final performance estimate. We report final models as the main results. Validation-based selection was also run for the other full-observation seeds and gives $91.1 \pm 1.6\%$ over five seeds (Table VII), higher and far less variable than the final models, which suggests that part of the seed-to-seed spread of the final models comes from where training happens to stop.

A. Reward Ablations

Table XII removes one component at a time and trains for 5×10^5 steps with two seeds, compared with the 500k checkpoints of the full-reward runs with the same seeds. The two full-reward seeds differ by 25 percentage points at 500k steps, more than any difference between variants, so none of the ablations shows an effect that can be separated from seed variation. In particular, removing the anti-oscillation term does not change coverage measurably, and point-sampled coverage trains as well as the swept test, consistent with Section XI. Resolving effects of this size would need many more seeds.

XVI. FAILURE ANALYSIS

A natural hypothesis is that concave regions account for most of the missed coverage. Measured over the 20 test episodes of the seed-42 model (Fig. 4), concave pixels (radius of curvature below 60 px, 496 pixels) are missed in 16.3% of episodes, against 13.6% for straight and 13.1% for convex parts. Concavity contributes, but only modestly. Most lost coverage comes from whole segments left untraced when the agent loops without progress and the episode is truncated, as in the worst episode of Fig. 2.

To check this across seeds, we pooled the 100 test episodes of the five full-observation final models and classified each target pixel by the sign of its smoothed curvature (bending toward the loop interior is convex, away from it is concave, and below a small curvature threshold is straight). This classification differs from the radius-based one above, so the numbers are not directly comparable (Table XIII). Concave pixels are missed in 14.4% of episodes, against 11.7% for straight and

TABLE IX
OBSERVATION ABLATIONS: FINAL-MODEL COVERAGE (%), MEAN ± SD OVER SEED MEANS; PATH1 COMPLETIONS ARE COUNTED OVER ALL SEEDS × 20 STARTS. HELD-OUT IS THE MEAN OVER TEN PROCEDURAL SHAPES.

Observation tier	Seeds	path1	Compl. (path1)	path2	path3	Held-out
Full	5	86.4 ± 4.7	5/100	49.9 ± 8.0	49.3 ± 9.0	79.8 ± 6.4
No-direction	5	86.4 ± 8.2	8/100	44.8 ± 7.1	49.7 ± 8.4	73.3 ± 19.7
No-direction, noisy	3	63.3 ± 21.0	0/60	40.6 ± 12.1	35.9 ± 4.1	43.8 ± 22.8
No-visual	3	94.1 ± 0.8	39/60	91.8 ± 4.3	90.2 ± 5.2	94.9 ± 0.2

TABLE X
CROSS-TARGET GENERALIZATION: COVERAGE (%), 20 TEST STARTS, $\phi = 0.95$. "HELD-OUT" IS THE MEAN ± SD OVER TEN PROCEDURAL SHAPES NOT USED IN TRAINING. PPO ROWS ARE FINAL MODELS. †SPEED TUNED ON VALIDATION STARTS. "–": NOT EVALUATED ON THE CURRENT HELD-OUT SHAPES (THE CHECKPOINT OF THE 64-SHAPE MODEL IS NOT AVAILABLE).

Method	path1	path2	path3	Held-out shapes
Greedy†	95.0	95.1	95.1	95.0 ± 0.0
Pure pursuit†	95.1	95.1	95.1	95.0 ± 0.0
PPO trained on path1 (seed 42)	86.2	56.6	50.5	70.9 ± 6.9
PPO trained on path1 (seed 1)	90.1	45.4	45.7	76.5 ± 5.3
PPO trained on path1 (seed 2)	80.3	59.7	62.8	84.4 ± 1.9
PPO trained on path1 (seed 3)	91.9	47.2	49.5	87.0 ± 2.1
PPO trained on path1 (seed 4)	83.8	40.4	38.2	80.1 ± 2.6
PPO trained on path1 (seed 2026, $\phi = 0.80$)	94.3	55.4	61.2	90.9 ± 0.9
PPO trained on 64 procedural shapes	53.8	58.6	69.3	–

TABLE XI
DETERMINISTIC CHECKPOINT SWEEP OF THE SEED-42 RUN (PATH1, COVERAGE %). THE 1000K ROW IS THE CHECKPOINT WRITTEN AT EXACTLY 1,000,000 STEPS; THE LAST ROW IS THE FINAL MODEL.

Steps	Validation	Test	Completed (test)
100k	36.5	28.0	0/20
200k	57.9	58.3	0/20
300k	72.3	77.9	0/20
400k	92.3	90.2	4/20
500k	90.1	91.6	6/20
600k	83.3	82.7	0/20
700k	87.9	88.2	1/20
800k	88.2	86.6	0/20
900k	86.3	87.3	1/20
1000k	84.3	84.3	0/20
Final (1,003,520)	–	86.2	0/20

TABLE XII
ABLATIONS ON PATH1 AFTER 500K STEPS: TEST COVERAGE (%) FOR SEEDS 1 / 2 AND THEIR MEAN

Variant	Seeds 1 / 2	Mean
Full reward (500k)	62.8 / 87.7	75.3
No anti-oscillation	81.5 / 71.9	76.7
No approach shaping	49.8 / 84.3	67.0
Point-sampled coverage	81.0 / 85.1	83.1

13.7% for convex parts, and 93.0% of all missed pixels belong to episodes ended by the stagnation limit.

The same analysis shows that the learned policies are not fast. Their mean speed is between 1.9 and 2.8 units/s across tiers and essentially none of their steps exceeds $0.9\,v_{\max}$. Loops and truncation are therefore not explained by top-speed overshoot. The no-visual agents leave much less to stagnation (19.4% of missed pixels) and miss about 5.5% to 6.0% of each curvature class, compared with 12% to 14% for the full observation, which is consistent with the image input being associated with, rather than preventing, the loops.

Why does removing the image help? (speculative.) The no-visual tier reaches 94.1% on path1 against 86.4% for the full observation. Of the 100 full-observation test episodes, 88 end by the stagnation limit and 5 complete; of the 60 no-visual episodes, only 9 end by stagnation, 12 run to the step limit while still making progress, and 39 complete. Mean speed is almost unchanged (2.67 versus 2.62 units/s, Table XIII), the lower miss rate holds in every curvature class, and the gap is larger on unseen open strokes (91.8% and 90.2% against 49.9% and 49.3%, Table IX). The no-visual agent keeps the numerical direction $\hat{u}$ and distance u_t and loses only the 64×64 crop, so the crop is associated with the loops; these data do not show why, and the seed sets differ (five versus three) with no significance test. We therefore offer three hypotheses that we have *not* tested. (i) *Overfitting to one contour:* the crop is a 4,096-value input, and with a single training target the policy may learn the appearance of path1 rather than a general feedback rule; this fits the large drop on path2 and path3 but does not by itself explain the loops on path1. (ii) *A local attractor:* the crop shows uncovered contour only within about 1.6 world units, and because the two edges of the stroke lie 8.4 px apart, tracing one edge leaves most of the other still visible (Section XI); a policy that steers toward what the crop shows, rather than toward the globally nearest uncovered pixel given by $\hat{u}$, could alternate between edges without gaining coverage until the stagnation limit ends the episode. (iii) *Optimisation:* by our count the convolutional branch holds most of the roughly 430,000 parameters, and

TABLE XIII
MECHANISM ANALYSIS ON `path1` (FINAL MODELS, POOLED TEST EPISODES). MISS RATE IS THE PERCENTAGE OF EPISODES IN WHICH A PIXEL OF THE CLASS WAS LEFT UNCOVERED. MEAN SPEED (WORLD UNITS/S) IS POOLED OVER ALL STEPS, SO IT DIFFERS SLIGHTLY FROM THE PER-EPISODE AVERAGES IN TABLE VIII.

Tier	Mean speed	Stagn. (%)	Concave	Straight	Convex
Full	2.67	93.0	14.4	11.7	13.7
No-direction	2.81	94.9	15.5	11.9	13.2
No-direction, noisy	1.93	100.0	35.9	33.9	38.0
No-visual	2.62	19.4	5.8	5.5	6.0

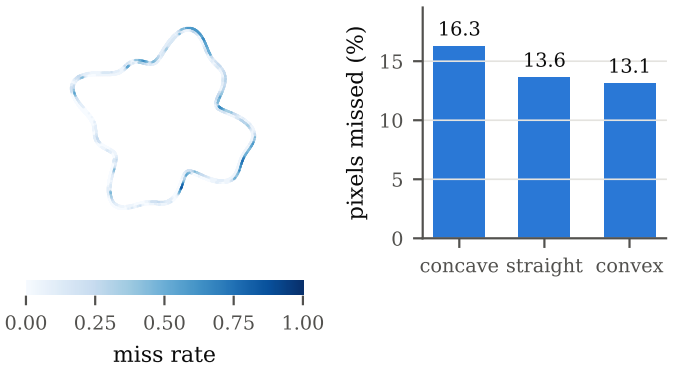


Fig. 4. Left: fraction of the 20 test episodes in which each target pixel of `path1` was left uncovered. Right: miss rate by local curvature.

its gradients may dominate or add noise to the shared 256-dimensional representation, slowing the learning of the 11-value feedback law that suffices for the task; we logged neither gradients nor training dynamics. These could be tested by evaluating the full-observation policies with the crop zeroed at test time, by retraining with a smaller or larger crop or with the convolutional branch frozen or regularised, by retraining on a single-edge target, and by matching seeds across tiers; we have not run these experiments.

XVII. DISCUSSION

The coverage formulation. The target-resolution coverage formulation measures the intended task more directly than proximity- or cell-based coverage: it cannot be inflated by revisiting, it tracks traced arc length up to a small end-cap bias, and under the reward built on it the degenerate policies we tried are unprofitable. The continuity term is not speed-invariant, as the slow-tracer test shows, so return should not be used to compare methods. The coverage tolerance couples nearby structures, so for thin strokes the metric measures stroke coverage rather than the reconstruction of each edge, and absolute coverage values depend strongly on ρ.

Learning behaviour. Continuous-control PPO learns meaningful tracing from a local visual observation and a compact state. Its best single run is close to the greedy controller in coverage, but it is not reliable: seed means of the final models span 80.3% to 91.9%, no seed completes the task from every start, and the policies transfer poorly to unseen open strokes. The observation ablations suggest that the image branch is not what produces the tracing. Removing the image improves coverage and completion, removing the direction features changes nothing, and only the noisy tier shows a clear loss.

Implications for benchmark design. The nearest-uncovered direction $\hat{u}$ is part of the agent's observation, and a simple velocity controller that uses this feature together with the bead's velocity solves the task at a moderate speed from every start on every target tested. Under this observation design the benchmark therefore does not isolate the contribution of learning: it measures whether a policy can match a well-damped feedback law, rather than whether learning is necessary, and in our experiments no learned policy exceeds that law. We regard this as a result about benchmark construction. It establishes the greedy controller as the reference bar that any learned contour tracer must clear. Our removal of $\hat{u}$ and u_t did not produce a setting in which learning is required, and the only harder setting we tried, sensing noise, has no controller baseline yet; it remains the natural next benchmark. The earlier conjecture that the reward favours fast motion, which the inertial dynamics turn into overshoot, is not supported: the learned policies run at about 2.6 units/s, well below $v_{\max}$. The cause of the loop-and-truncate failures is not resolved by the present experiments; Section XVI lists untested hypotheses.

XVIII. LIMITATIONS AND FUTURE WORK

All results are in simulation. The main comparison uses five seeds for the full and no-direction tiers, three seeds for the noisy and no-visual tiers, and each reward-ablation cell two. The $\phi = 0.80$ result is a single run (seed 2026); an earlier $\phi = 0.80$ run whose seed was not recorded agrees on coverage but not on the number of completions (13 versus 9 of 20), so we do not draw conclusions about the threshold from either. The model trained on 64 procedural shapes is also a single run, and its checkpoint is not in the released repository, so it has no held-out evaluation. The held-out shapes are one family of smooth closed curves; noisy, fragmented and self-intersecting images are untested. The coverage tolerance ρ was fixed at 8 px for training and evaluation; Table V reports how the coverage of existing trajectories changes with ρ, but no policy was retrained at another tolerance. PPO hyperparameters were not tuned beyond the original configuration, and other algorithms such as SAC and TD3 were not compared.

The most useful next steps are: testing the explanations for the effect of the image input given in Section XVI; evaluating the greedy controller, or a filtered variant of it, under the same sensing noise as the noisy tier so that the noisy setting has a baseline; training on larger and more varied contour sets; testing noisy, fragmented and self-intersecting contours; penalizing time more directly than through discounting; retraining at other coverage tolerances; comparing SAC and TD3 under the same protocol; and validating on a physical platform.

XIX. CONCLUSION

This paper presented a continuous-control reinforcement-learning framework for reconstructing contours extracted from

images, combining contour extraction, distance-transform observations, an inertial bead simulation and PPO-based control. Its main methodological contribution is a swept target-resolution coverage metric kept in a per-episode Boolean mask, which cannot increase through repeated traversal and tracks traced arc length on an analytic target. Under the reward built on it, raster sweeping, stationary behaviour and off-contour motion are unprofitable, although a slow-tracer test shows that the composite reward can be inflated through its continuity term.

Evaluated against simple controllers under one protocol, the PPO agent learns substantial tracing behaviour, and its best run approaches the greedy controller's coverage. Results vary widely across runs, however, the learned policies transfer poorly to unseen open strokes, and a greedy controller that uses one of the agent's own input features is more reliable. Observation ablations show that removing that feature leaves PPO unchanged, that removing the image makes it better, and that only added sensing noise clearly degrades it. We therefore propose the greedy controller as the reference bar for learned contour tracers, and report that the present observation design does not require learning to reach it. A setting in which learning is needed, such as noisy sensing against a matched controller baseline, is left for future work. The fully specified protocol allows future learned tracers to be compared against that bar.

REFERENCES

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.

[2] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[3] M. Towers et al., "Gymnasium: A standard interface for reinforcement learning environments," arXiv preprint arXiv:2407.17032, 2024.

[4] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. International Conference on Learning Representations (ICLR)*, 2016.

[5] H. Choset, "Coverage for robotics—A survey of recent results," *Annals of Mathematics and Artificial Intelligence*, vol. 31, pp. 113–126, 2001.

[6] H. Choset and P. Pignon, "Coverage path planning: The boustrophedon cellular decomposition," in *Field and Service Robotics*, Springer, 1998.

[7] E. Galceran and M. Carreras, "A survey on coverage path planning for robotics," *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1258–1276, 2013.

[8] M. Theile, H. Bayerlein, R. Nai, D. Gesbert, and M. Caccamo, "UAV coverage path planning under varying power constraints using deep reinforcement learning," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 1444–1449.

[9] E. Garrido-Castaneda, J. A. Vasquez, and J. Antonio-Cruz, "Reinforcement learning-based coverage path planning," *Sensors*, vol. 25, no. 5, Art. 1592, 2025.

[10] H. Chen, W. Lu, X. Cui, S. Luo, and J. Zheng, "Re-DQN: Deep reinforcement learning for mowing-robot coverage path planning," *Sensors*, vol. 25, no. 2, Art. 416, 2025.

[11] A. Devo, G. Mao, G. Costante, and G. Loianno, "Autonomous single-image drone exploration with deep reinforcement learning and mixed reality," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5031–5038, 2022.

[12] A. Jonnarth, J. Zhao, and M. Felsberg, "Learning coverage paths in unknown environments with deep reinforcement learning," in *Proc. 41st International Conference on Machine Learning (ICML)*, PMLR, vol. 235, 2024, pp. 22491–22508.

[13] A. Jonnarth, O. Johansson, J. Zhao, and M. Felsberg, "Sim-to-real transfer of deep reinforcement learning agents for online coverage path planning," *IEEE Access*, vol. 13, pp. 106883–106905, 2025.

[14] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and characterizing reward hacking," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 9460–9471, 2022.

[15] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," arXiv preprint arXiv:1606.06565, 2016.

[16] V. Krakovna et al., "Specification gaming: The flip side of AI ingenuity," DeepMind Blog, 2020.

[17] Y. Ganin, T. Kulkarni, I. Babuschkin, S. M. A. Eslami, and O. Vinyals, "Synthesizing programs for images using reinforced adversarial learning," in *Proc. International Conference on Machine Learning (ICML)*, 2018.

[18] Z. Huang, W. Heng, and S. Zhou, "Learning to paint with model-based deep reinforcement learning," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[19] T. Zhou et al., "Learning to sketch with deep Q networks and demonstrated strokes," arXiv preprint arXiv:1810.05977, 2018.

[20] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon University, Tech. Rep. CMU-RI-TR-92-01, 1992.

[21] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[22] S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985.

[23] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[24] G. Borgefors, "Distance transformations in digital images," *Computer Vision, Graphics, and Image Processing*, vol. 34, no. 3, pp. 344–371, 1986.

[25] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance transforms of sampled functions," *Theory of Computing*, vol. 8, pp. 415–428, 2012.

[26] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. International Conference on Machine Learning (ICML)*, 1999.